## Coding Etiquette

**Readable code is not an optional extra. It is part of the solution.** These rules are intended to make your code easy to read, explain, debug, and improve.

### 1. Write readable and consistent code

Your code must be easy to read and understand by you, your colleagues, and your teachers.

- Indent and format the code properly.
- Use one statement per line.
- Use spaces after commas and around operators.
- Use braces consistently.
- Keep the same naming and formatting style throughout the program.
- Prefer a few clear lines to one complicated expression.

#### Good

```
if (student_count == 0) {
    return 0.0;
}

total_grade = total_grade + students[current_student].grade;
```

#### Avoid

```
if(student_count==0)return 0.0;
total_grade+=students[current_student].grade;
```

Comments should explain a decision, an assumption, or a non-obvious step. They should not simply repeat what the next instruction already says.

### 2. Use meaningful English names

Variables, functions, parameters, types, and comments must be written in English.

Names must describe the purpose of the element they identify.

Use the C naming conventions consistently:

| Element | Convention | Example |
|---|---|---|
| Variables, parameters, files and functions | `lower_snake_case` | `student_count` |
| Constants and macros | `UPPER_SNAKE_CASE` | `MAX_STUDENTS` |

| Element | Convention | Example |
| --- | --- | --- |
| Struct and enum types | `PascalCase` | `Student` |

**Good**

```c
int student_count;
double total_grade;
char course_name[50];
```

**Avoid**

```c
int i;
double tmp;
char str[50];
```

Function names should clearly describe the operation they perform.

**Good**

```c
double compute_average_grade(const Student *students, int student_count);
```

**Avoid**

```c
double calcOutput(Student *data, int n);
```

### 3. Declare things in the right place

Declarations should appear where they logically belong.

A C source file should normally follow this order:

1. `#include` directives;
2. constants and type definitions;
3. function prototypes;
4. `main`;
5. function definitions.

Inside a function:

- declare variables close to their first use;
- initialize variables before reading them;
- keep each variable in the smallest reasonable scope.

Follow the initial file organization provided with the exercise unless there is a clear reason to change it.

### 4. Write short and focused functions

Each function should perform one clear task.

This is the **Single-Responsibility Principle (SRP)**: a single function should have only one main responsibility.

#### Good

```c
double compute_average_grade(const Student *students, int student_count);
void print_students(const Student *students, int student_count);
```

#### Avoid

```c
double compute_average_and_print_students(Student *students, int
 ↪  student_count);
```

Avoid:

- **duplicated code**;
- functions **longer than approximately 60 lines** of code; if a function becomes longer, divide it into meaningful helper functions;
- more than **three nested control-flow** blocks, such as `if`, `for`, `while`, or `switch`; use guard clauses or helper functions to simplify the control flow;
- functions that combine different responsibilities.

Keep the algorithm separate from input and output operations.

### 5. Avoid magic numbers

A number that represents a limit, a rule, or a property of the problem should have a meaningful name.

#### Good

```c
#define MAX_STUDENTS 100

static const double PASSING_GRADE = 18.0;

if (grade >= PASSING_GRADE) {
    /* ... */
}
```

**Avoid**

```c
if (grade >= 18.0) {
    /* Why 18.0? */
}
```

Not every numeric literal needs a name. Values such as `0` or `1` may remain directly in the code when their meaning is obvious.

### 6. Prefer simple and standard C

Use the simplest C construct that expresses the solution clearly.

Do not use advanced language features only to make the code shorter or more sophisticated.

Unless they are required by the exercise or explicitly introduced in class, avoid:

- complex macros;
- compiler-specific `#pragma` directives;
- unnecessary conditional compilation;
- non-standard compiler extensions;
- obscure pointer tricks;
- unnecessary casts;
- overly generic abstractions.

Clear and explicit code is preferable to clever code.

### 7. Use memory and resources correctly

When using arrays, dynamic memory, or files:

- initialize variables before using them;
- stay within array bounds;
- check whether memory allocation succeeds;
- check whether a file is opened successfully;
- release dynamically allocated memory when it is no longer needed;
- close every file that has been opened;
- never access memory after it has been released.

A program that produces the expected output but incorrectly handles memory or resources is not a correct solution.

### 8. Use the debugger to understand runtime behavior

The debugger is the primary and appropriate tool for analyzing the current state of a program and understanding its behavior at runtime. It makes it possible to inspect control flow, variables, memory, and data structures without changing the program's expected output.

Logging serves a different purpose: it records intentional runtime information that is part of the program's design. It does not replace debugging. Scattered diagnostic `printf` statements pollute the code when they are not removed and may also alter the output required by the specification.

### 9. Check your code before finishing

Before considering an exercise complete, verify that:

- ☐ the code is easy to read;
- ☐ names are meaningful and consistent;
- ☐ each function has one clear responsibility;
- ☐ meaningful values use named constants;
- ☐ the program compiles without warnings;
- ☐ there is no unused, commented-out, or debug code;
- ☐ memory and other resources are correctly released;
- ☐ the provided tests pass, when tests are included in the exercise;
- ☐ the program produces the expected output for the supplied examples.